浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

# Python II: Working with Numbers

## Composite Data Types

IBI 1, Lecture 6.2

Ying CHI – yingchi@intl.zju.edu.cn

Adapted from Rob

Semester 2, 2025/26

# This lecture is a Python reference guide

"They're encyclopedias, Timmy. . . they're an early form of

- The information here will help you write code, but its best to practice – not to memorise.
- You can often achieve the same thing by different means, and you should choose what works best for you (not me!).

# Learning objectives

## After this lecture, you should be able to:

- Work with arrays and lists in Python.
- Describe data collections in Python.

# Outline

1. List – add/remove/sort
2. Ndarray – N-dimensional array object
3. Dictionary – student information/keyword frequency

# Lists

- A list is an **ordered sequence** of information, accessible by index – its place in the lists.

- A list is denoted by **square brackets**, [ ].

- A list contains elements:
  - usually homogeneous (i.e. all integers)
  - can contain mixed types (rare)

- List elements can be changed, so a list is **mutable**.

# Indices and ordering

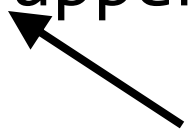
```
a_list = [] empty list
L = [2, 'a', 4, 1]
len(L)          # evaluates to 4
L[0]            # evalutes to 2
L[2] + 1        # evaluates to 5
L[4]            # gives an error
i=2
L[i-1]          # evaluates to 'a' since L[1] = 'a'
```

# Changing elements

- Lists are **mutable**!
- Assigning an element at an index (position) changes the value:
- `L = [2, 1, 3]`
- `L[1] = 5`
- `print(L)` # returns 2,5,3 – note this is the **same object** L
- `print(L[1:])` # returns 5,3 – here I have sliced the first element
- `L = L[1:]`
- `print(L)` # what does this return?

# Add items to a list (1)

- **Add** elements to the end of a list with L.append(element)
- **Mutates** the list!
- `L = [2,1,3]`
- `L.append(5)` # returns `[2,1,3,5]`



- What is the dot?
- Lists are Python variables, everything in Python is a variable.
- Variables store information
- Variables have methods and functions
- Functions can be accessed by `object_name.do_something()`

# Add items to a list (2)

- To combine lists together use the + operator (**concatenation**) to give you a new list.
- **Mutate** list with L.extend(list)

```
L1 = [2,1,3]
L2 = [4,5,6]
L3 = L1 + L2  # L1 and  L2 unchanged; L3 is [2,1,3,4,5,6]
L1.extend(L2)  # mutated L1 to [2,1,3,4,5,6]
```

Can you see the difference between these?

# Removing items from a list

- Delete element at a specific location with del(L[index])
- Remove element at end of list with L.pop(), returns to removed element.
- Remove a specific element with L.remove(element)
  - Looks for the element and removes it
  - If the element occurs multiple times, removes the first occurrence.
  - If element not in list, gives an error.

```
L = [2,1,3,6,3,7,0]
L.remove(2) # mutates L = [1,3,6,3,7,0]
L.remove(3) # mutates L = [1,6,3,7,0]
del(L[1]) # mutates L = [1,3,7,0]
L.pop() # returns 0 and mutates L = [1,3,7]
```

can you see how all these work?

# Sorting lists

- `sort() and sorted()`
- `reverse()`
- `L = [9,6,0,3]`
- `sorted(L)` # returns sorted list, does not mutate L
- `L = sorted(L)`
- `L.sort()` # mutates L = [0,3,6,9]
- `L.reverse()` # mutates L=[9,6,3,0]

which do you prefer?

- And many more!
- https://docs.python.org/3/tutorial/datastructures.html

# For-loops: Iterating over a list

- Compute the **sum of elements** of a list
- Common pattern, iterate over list elements

```
total = 0
for i in range(len(L)):
    total += L[i]
print total
```

```
total = 0
for i in L:
    total += i
print total
```

which do you prefer?

- Notice:
  - List elements are indexed 0 to len(L)-1
  - range(n) goes from 0 to n-1

# Mutation and iteration

- **Avoid** mutating a list as you are iterating over it:

```
def remove_dups(L1, L2):
    for e in L1:
        if e in L2:
            L1.remove(e)
```

- `L1 = [1,2,3,4]`
- `L2 = [1,2,5,6]`
- `remove_dups(L1, L2)`
- `L1` is `[2,3,4]` not `[3,4]` → why?

# Mutation and iteration

- **Avoid** mutating a list as you are iterating over it:

```
def remove_dups(L1, L2):
    for e in L1:
        if e in L2:
            L1.remove(e)
```

```
def remove_dups(L1, L2):
    L1_copy = L1[:]
    for e in L1_copy:
        if e in L2:
            L1.remove(e)
```

- `L1 = [1,2,3,4]`
- `L2 = [1,2,5,6]`
- `remove_dups(L1, L2)`
- Python uses an internal counter to keep track of the position it is in the loop
- Mutating changes the list length but Python doesn't update the counter
- The loop never sees element 2!

# Mutation and iteration

- **Avoid** mutating a list as you are iterating over it:

```
def remove_dups(L1, L2):
    for e in L1:
        if e in L2:
            L1.remove(e)
```

```
def remove_dups(L1, L2):
    L1_copy = L1[:]
    for e in L1_copy:
        if e in L2:
            L1.remove(e)
```

- `L1 = [1,2,3,4]`
- `L2 = [1,2,5,6]`
- `remove_dups(L1, L2)`
- Python uses an internal counter to keep track of the position it is in the loop
- Mutating changes the list length but Python doesn't update the counter
- The loop never sees element 2!
- What is L1 now?

# Outline

1. List – add/remove/sort
2. Ndarray – N-dimensional array object
3. Dictionary – student information/keyword frequency

# ndarray

- **Numpy** (from the previous lecture) contains a powerful N-dimensional array object, here we will only consider 1-dimension.
- Ndarray is a list of elements, all of the **same type**.

```
Import numpy as np
array = np.array([1,2,3])
a = array[1]
```

- What is a? How would you extract '3' from this array?

# ndarray

- You can perform numerical operations on ndarray.
- ndarray is more efficient than list!

```
import numpy as np
a = np.array([1,2])
b = np.array([3,4])
a + b # returns [4,6]
```

- What is a*2, b**2, a*b?

# ndarray

- You can perform numerical operations on ndarray.
- ndarray is more efficient than list!

```
import numpy as np
a = np.array([1,2])
b = np.array([3,4])
a + b # returns [4,6]
a*2 # returns [2,4]
b**2 # returns [9,16]
a*b # returns [3,8]
```

# Outline

1. List – add/remove/sort
2. Ndarray – N-dimensional array object
3. Dictionary – student information/keyword frequency

# How to store gene info

- So far, we can store information in separate lists.

- `genes = ['Shh', 'NBAS', 'Dlx5', 'PTEN']`
- `lengths=[9410,3944141,4442,105338]`

- We need a **separate list** for each item.
- Each list must have the **same length**.
- Info stored across lists must be at the **same index**, where each index refers to info for a different gene.

# Dictionaries

- It's nice (easier!) to index items of interest directly (not always integers).
- It's nice (easier!) to use one data structure, instead of separate lists.

A list

| | |
|---|---|
| 0 | element 1 |
| 1 | element 2 |
| 2 | element 3 |
| 3 | element 4 |
| … | … |

index

element

A dictionary

| | |
|---|---|
| key 1 | value 1 |
| key 2 | value 2 |
| key 3 | value 3 |
| key 4 | value 4 |
| … | … |

custom index by label

element

# Python dictionary

- Dictionaries store data as key:value pairs:

| | |
|---|---|
| Shh | 9410 |
| NBAS | 3944141 |
| Dlx5 | 4442 |
| PTEN | 105338 |

empty dictionary

custom index by label

element

```
my_dict = {}
genes = {'Shh':9410, 'NBAS':3944141, 'Dlx5':4442, 'PTEN':105338}
```

Key 1 Value 1 Key 2 Value 2 Key 3 Value 3 Key 4 Value 4

# Dictionary lookup

- Similar to indexing in a list
- Looks up the key
- Returns the value associated with the key
- If key isn't found, get an error.

| | |
|---|---|
| Shh | 9410 |
| NBAS | 3944141 |
| Dlx5 | 4442 |
| PTEN | 105338 |

custom index by label

element

```
genes = {'Shh':9410, 'NBAS':3944141, 'Dlx5':4442, 'PTEN':105338}
genes['Dlx5'] # evaluates to 4442.
genes['TP53'] # gives a KeyError
```

**Note the different brackets!**

# Dictionary operations

```
genes = {'Shh':9410, 'NBAS':3944141, 'Dlx5':4442, 'PTEN':105338}
```

- **Add** an entry

```
genes['CTCF']=76779
```

- **Test** if key in dictionary

```
'NBAS' in genes # returns True
'NR3C1' in genes # returns False
```

- **Delete** entry

```
del(genes['Dlx5'])
```

| | |
|---|---|
| Shh | 9410 |
| NBAS | 3944141 |
| Dlx5 | 4442 |
| PTEN | 105338 |

# Dictionary keys and values

- Values:
  - Can be any type (**immutable** and **mutable**)
  - Can **duplicate**
  - Dictionary values can be lists, even other dictionaries!

- Keys:
  - Must be **unique**
  - **Immutable** type (int, float, string, boolean)
  - Be careful with floats!

- There is **no order** to keys or values!

| | |
|---|---|
| Shh | 9410 |
| NBAS | 3944141 |
| Dlx5 | 4442 |
| PTEN | 105338 |

custom index by label

element

```
genes = {'Shh':9410, 'NBAS':3944141, 'Dlx5':4442, 'PTEN':105338}
```

Key 1 Value 1 Key 2 Value 2 Key 3 Value 3 Key 4 Value 4

# Lists vs Dictionaries

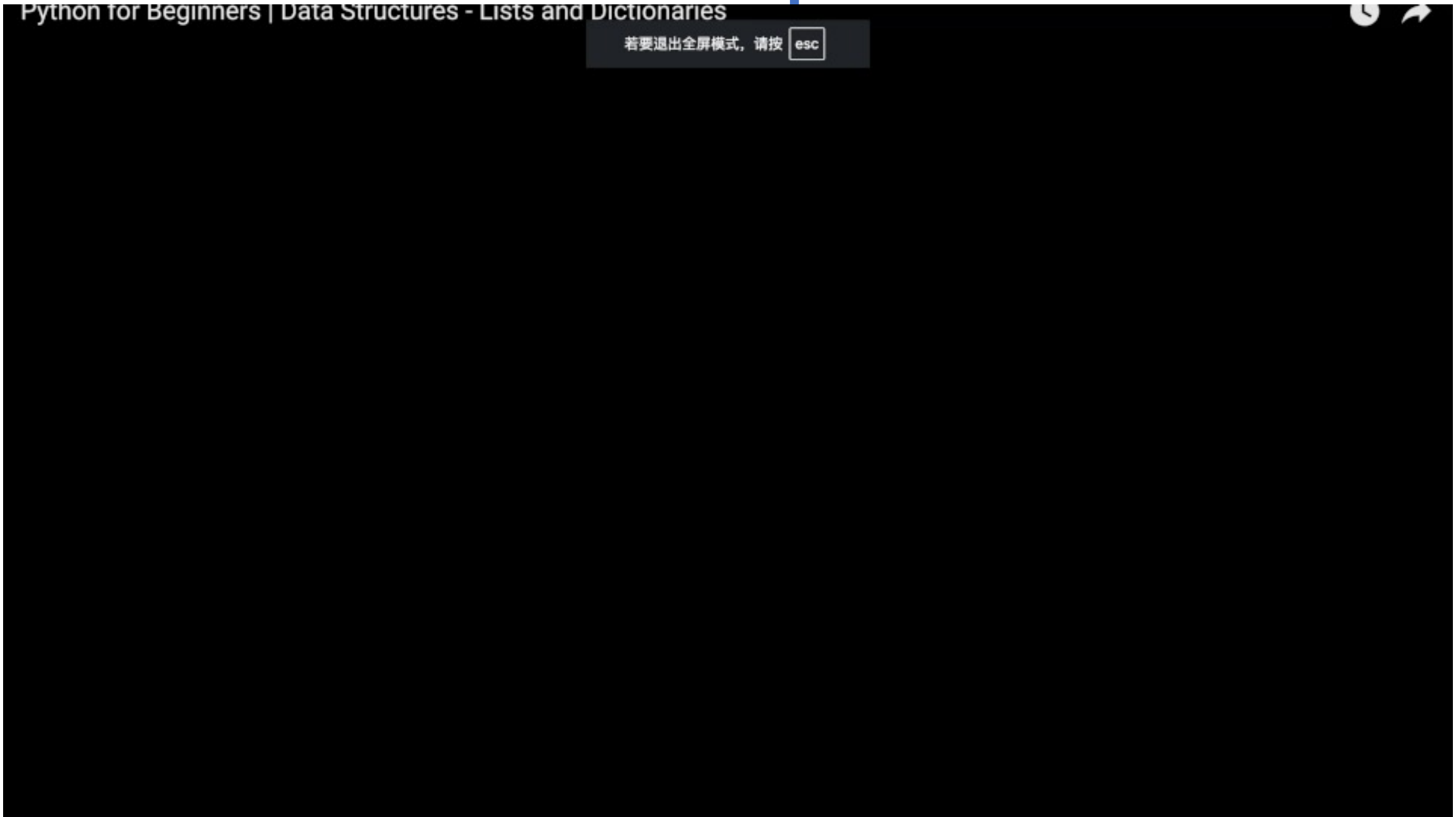
**What do you think are the similarities and differences?**

# Lists vs Dictionaries

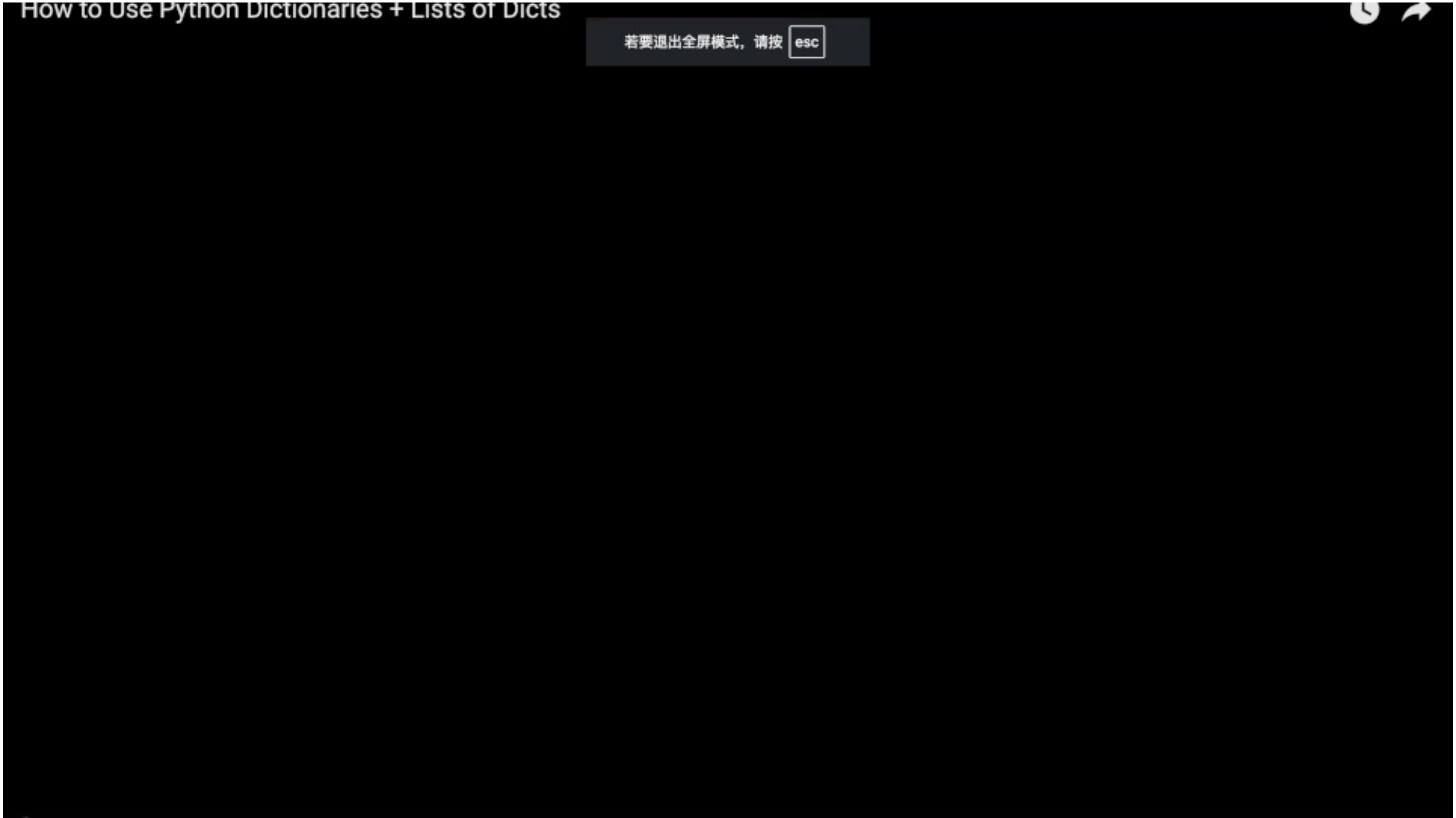
- **Ordered** sequence of elements
- Look up elements by an integer index
- Indices have an **order**
- Index is an **integer**

- **Matches** "keys" to "values"
- Look up one item by another item
- **No order** is guaranteed
- Key can be any **immutable** type

# Extension in practices



# List of Dictionaries



# Packages comparison: NumPy vs. Pandas

# Learning objectives

## Now, you should be able to:

- Work with arrays and lists in Python.
- Describe data collections in Python.

浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

# Python II: Working with Numbers

## Composite Data Types

## Any questions?

IBI 1, Lecture 6.2

Ying CHI – yingchi@intl.zju.edu.cn

Semester 2, 2024/25